| EX.NO: 14 | TIC-TAC-TOE |
|---|---|
| 20/03/2026 | |

## AIM

To Develop Tic Tac Toe Game using MQTT Broker.

## APPARATUS REQUIRED

- ESP 32 Dev Module

## PRE-CONDITIONS

- Mosquitto MQTT broker running and reachable on the network.
- ESP32 devices flashed with Wi-Fi credentials for both players (if using two devices).
- Input method (touch/display or buttons) configured and tested.

## POST-CONDITIONS

- Game moves are exchanged over MQTT and board state stays synchronized between players.
- Game correctly detects win/draw conditions and publishes final result; verify by playing a full game and observing MQTT messages/serial output.

## CODE

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <map>

//   Network
const char* WIFI_SSID     = "Wokwi-GUEST";  // Wokwi virtual WiFi
const char* WIFI_PASSWORD = "";

const char* MQTT_HOST     = "100.64.0.1";
const int   MQTT_PORT     = 1883;
const char* MQTT_CLIENT   = "esp32-ttt-server";

//   Structs
struct GameSession {
  char    board[9];        // ' '=empty  'X'=X  'O'=O
  String  player1;         // pid of creator (always X)
  String  player2;         // pid of opponent, or "AI"
  String  currentTurn;    // pid whose turn it is, "AI" when AI thinking
```

```
  bool    aiMode;
  bool    active;
};

struct Player {
  String name;
  String client;  // "web" or "flutter"
};

//   State
std::map<String, GameSession> sessions;
std::map<String, Player>      players;

WiFiClient    wifiClient;
PubSubClient  mqtt(wifiClient);
unsigned long lastHeartbeat = 0;

//
//  Helpers
//

String genId(int len = 4) {
  const char hex[] = "0123456789abcdef";
  String id = "";
  for (int i = 0; i < len; i++) id += hex[random(16)];
  return id;
}

//   Minimax AI

int winLines[8][3] = {
  {0,1,2},{3,4,5},{6,7,8},   // rows
  {0,3,6},{1,4,7},{2,5,8},   // cols
  {0,4,8},{2,4,6}            // diags
};

int evaluate(char b[9]) {
  for (auto& w : winLines) {
    if (b[w[0]] != ' ' && b[w[0]] == b[w[1]] && b[w[1]] == b[w[2]])
      return b[w[0]] == 'O' ? 10 : -10;
  }
  return 0;
}

bool boardFull(char b[9]) {
  for (int i = 0; i < 9; i++) if (b[i] == ' ') return false;
  return true;
}
```

```
int minimax(char b[9], int depth, bool isMax, int alpha, int beta) {
  int score = evaluate(b);
  if (score != 0)     return score - (isMax ? depth : -depth);
  if (boardFull(b))   return 0;

  if (isMax) {
    int best = -100;
    for (int i = 0; i < 9; i++) {
      if (b[i] == ' ') {
        b[i] = 'O';
        best = max(best, minimax(b, depth + 1, false, alpha, beta));
        b[i] = ' ';
        alpha = max(alpha, best);
        if (beta <= alpha) break;
      }
    }
    return best;
  } else {
    int best = 100;
    for (int i = 0; i < 9; i++) {
      if (b[i] == ' ') {
        b[i] = 'X';
        best = min(best, minimax(b, depth + 1, true, alpha, beta));
        b[i] = ' ';
        beta = min(beta, best);
        if (beta <= alpha) break;
      }
    }
    return best;
  }
}

int bestAiMove(char b[9]) {
  int bestVal = -100, bestCell = -1;
  for (int i = 0; i < 9; i++) {
    if (b[i] == ' ') {
      b[i] = 'O';
      int val = minimax(b, 0, false, -100, 100);
      b[i] = ' ';
      if (val > bestVal) { bestVal = val; bestCell = i; }
    }
  }
  return bestCell;
}

//   Win detection
```

```
// Returns  ""          : game ongoing
// Returns  "draw"      : board full, no winner
// Returns  "X:0,4,8"   : X wins, winning cell indices
String checkResult(char b[9]) {
  for (auto& w : winLines) {
    if (b[w[0]] != ' ' && b[w[0]] == b[w[1]] && b[w[1]] == b[w[2]]) {
      char sym = b[w[0]];
      String s = String(sym) + ":" +
                 String(w[0]) + "," +
                 String(w[1]) + "," +
                 String(w[2]);
      return s;
    }
  }
  return boardFull(b) ? "draw" : "";
}

//
//  MQTT Publishers
//

void pubState(const String& gid, GameSession& g) {
  StaticJsonDocument<512> doc;
  JsonArray board = doc.createNestedArray("board");
  for (int i = 0; i < 9; i++)
    board.add(g.board[i] == ' ' ? "" : String(g.board[i]));

  doc["gid"]      = gid;
  doc["turn"]     = g.currentTurn;
  doc["p1"]       = g.player1;
  doc["p2"]       = g.player2;
  doc["ai_mode"]  = g.aiMode;
  doc["status"]   = "ongoing";

  String payload, topic = "ttt/game/" + gid + "/state";
  serializeJson(doc, payload);
  mqtt.publish(topic.c_str(), payload.c_str(), true);
  Serial.println("[PUB] " + topic);
}

void pubResult(const String& gid, const String& result, GameSession& g) {
  StaticJsonDocument<256> doc;
  String topic = "ttt/game/" + gid + "/result";

  if (result == "draw") {
    doc["winner"]        = "draw";
    doc["winner_symbol"] = "";
    doc["winning_cells"] = "";
```

```
  } else {
    String sym   = result.substring(0, 1);
    String cells = result.substring(2);
    String winner = (sym == "X") ? g.player1 : g.player2;
    doc["winner"]        = winner;
    doc["winner_symbol"] = sym;
    doc["winning_cells"] = cells;
  }

  doc["p1"] = g.player1;
  doc["p2"] = g.player2;

  String payload;
  serializeJson(doc, payload);
  mqtt.publish(topic.c_str(), payload.c_str(), true);
  Serial.println("[PUB] " + topic + " result=" + result);

  g.active = false;
}

void pubOnlinePlayers() {
  StaticJsonDocument<1024> doc;
  JsonArray arr = doc.createNestedArray("players");
  for (auto& kv : players) {
    JsonObject o = arr.createNestedObject();
    o["pid"]    = kv.first;
    o["name"]   = kv.second.name;
    o["client"] = kv.second.client;
  }
  String payload;
  serializeJson(doc, payload);
  mqtt.publish("ttt/sys/players/online", payload.c_str(), true);
}

void pubCreated(const String& pid, const String& gid,
                const String& sym, const String& opponentPid,
                const String& opponentName, bool aiMode) {
  StaticJsonDocument<256> doc;
  doc["gid"]           = gid;
  doc["symbol"]        = sym;
  doc["opponent_pid"]  = opponentPid;
  doc["opponent_name"] = opponentName;
  doc["ai_mode"]       = aiMode;

  String payload, topic = "ttt/game/" + pid + "/created";
  serializeJson(doc, payload);
  mqtt.publish(topic.c_str(), payload.c_str());
}
```

```
//
//  Move handling (shared by human + AI)
//

void applyMoveAndRespond(const String& gid, GameSession& g, int cell,
 ↪  char sym) {
  g.board[cell] = sym;
  String result = checkResult(g.board);

  if (result != "") {
    pubState(gid, g);
    pubResult(gid, result, g);
    return;
  }

  if (g.aiMode) {
    // AI's turn
    g.currentTurn = "AI";
    pubState(gid, g);

    delay(400);  // brief pause so clients see "AI thinking"

    int aiCell = bestAiMove(g.board);
    g.board[aiCell] = 'O';
    g.currentTurn   = g.player1;

    String aiResult = checkResult(g.board);
    pubState(gid, g);
    if (aiResult != "") pubResult(gid, aiResult, g);
  } else {
    g.currentTurn = (g.currentTurn == g.player1) ? g.player2 : g.player1;
    pubState(gid, g);
  }
}

//
//  MQTT Callback
//

void onMessage(char* topicC, byte* payload, unsigned int len) {
  String topic = String(topicC);
  String msg   = "";
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];

  Serial.println("[SUB] " + topic + " | " + msg);

  StaticJsonDocument<512> doc;
```

```
if (deserializeJson(doc, msg) != DeserializationError::Ok) {
  Serial.println("  JSON parse error");
  return;
}

//   ttt/game/new
if (topic == "ttt/game/new") {
  String pid  = doc["pid"].as<String>();
  String mode = doc["mode"].as<String>();   // "ai" | "human"
  String name = doc["name"].as<String>();

  String gid = genId(4);
  GameSession g;
  for (int i = 0; i < 9; i++) g.board[i] = ' ';
  g.player1     = pid;
  g.player2     = (mode == "ai") ? "AI" : "";
  g.currentTurn = pid;
  g.aiMode      = (mode == "ai");
  g.active      = true;
  sessions[gid] = g;

  pubCreated(pid, gid, "X", "AI", "AI", g.aiMode);
  pubState(gid, sessions[gid]);
  Serial.println("  Created session " + gid + " mode=" + mode);
  return;
}

//   ttt/game/{gid}/move
if (topic.startsWith("ttt/game/") && topic.endsWith("/move")) {
  // Extract gid: "ttt/game/XXXX/move" → skip 9, trim 5
  String gid = topic.substring(9, topic.length() - 5);

  if (sessions.find(gid) == sessions.end()) {
    Serial.println("  Unknown session: " + gid); return;
  }
  GameSession& g = sessions[gid];
  if (!g.active) { Serial.println("  Session inactive"); return; }

  String pid = doc["player"].as<String>();
  int    cell = doc["cell"].as<int>();

  // Validate
  if (pid != g.currentTurn) {
    Serial.println("  Wrong turn. Expected: " + g.currentTurn + " got:
↪ " + pid);
    return;
  }
  if (cell < 0 || cell > 8 || g.board[cell] != ' ') {
```

```
    Serial.println("  Invalid cell " + String(cell));
    return;
  }

  char sym = (pid == g.player1) ? 'X' : 'O';
  applyMoveAndRespond(gid, g, cell, sym);
  return;
}

//   ttt/lobby/announce
if (topic == "ttt/lobby/announce") {
  String pid    = doc["pid"].as<String>();
  String name   = doc["name"].as<String>();
  String client = doc["client"].as<String>();
  players[pid]  = { name, client };
  Serial.println("  Player online: " + name + " (" + pid + ")");
  pubOnlinePlayers();
  return;
}

//   ttt/lobby/leave
if (topic.startsWith("ttt/lobby/leave/")) {
  String pid = topic.substring(16);
  players.erase(pid);
  pubOnlinePlayers();
  return;
}

//   ttt/lobby/invite/{target_pid}
// Clients subscribe directly to their own pid topic;
// ESP32 just validates the sender is a known player.
if (topic.startsWith("ttt/lobby/invite/")) {
  // The broker delivers this directly to subscribers.
  // ESP32 doesn't need to relay - just log it.
  String targetPid = topic.substring(17);
  String fromPid   = doc["from_pid"].as<String>();
  Serial.println("  Invite: " + fromPid + " -> " + targetPid);
  return;
}

//   ttt/lobby/response/{inviter_pid}
if (topic.startsWith("ttt/lobby/response/")) {
  String inviterPid    = topic.substring(19);
  bool   accepted      = doc["accepted"].as<bool>();
  String responderPid  = doc["pid"].as<String>();
  String responderName = doc["name"].as<String>();

  if (!accepted) {
```

```
      // Tell inviter they were declined
      StaticJsonDocument<128> d;
      d["from_pid"]  = responderPid;
      d["from_name"] = responderName;
      d["accepted"]  = false;
      String dp, dt = "ttt/game/" + inviterPid + "/created";
      serializeJson(d, dp);
      mqtt.publish(dt.c_str(), dp.c_str());
      return;
    }

    // Both accepted → create PvP session
    String gid = genId(4);
    GameSession g;
    for (int i = 0; i < 9; i++) g.board[i] = ' ';
    g.player1     = inviterPid;
    g.player2     = responderPid;
    g.currentTurn = inviterPid;
    g.aiMode      = false;
    g.active      = true;
    sessions[gid] = g;

    // Notify inviter (X)
    String inviterName = players.count(inviterPid) ?
↪   players[inviterPid].name : inviterPid;
    pubCreated(inviterPid, gid, "X", responderPid, responderName, false);
    // Notify responder (O)
    pubCreated(responderPid, gid, "O", inviterPid, inviterName, false);

    pubState(gid, sessions[gid]);
    Serial.println("  PvP session " + gid + ": " + inviterPid + " vs " +
↪   responderPid);
    return;
  }
}

//
//  Setup helpers
//

void connectWiFi() {
  Serial.print("[WiFi] Connecting to " + String(WIFI_SSID));
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println("\n[WiFi] Connected: " + WiFi.localIP().toString());
}
```

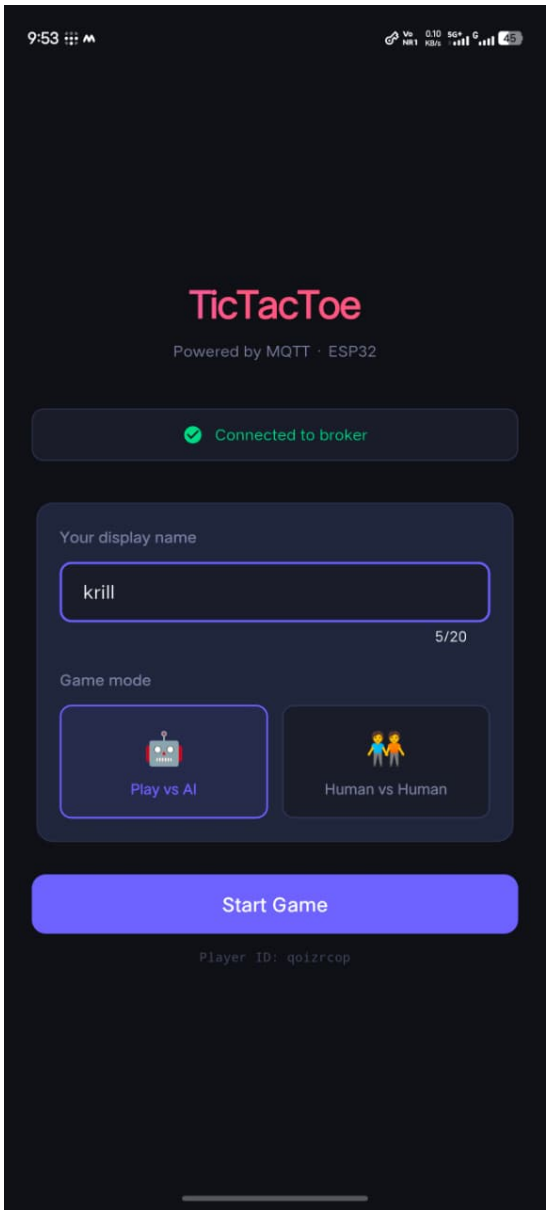
```
void connectMQTT() {
  while (!mqtt.connected()) {
    Serial.print("[MQTT] Connecting to " + String(MQTT_HOST) + "...");
    if (mqtt.connect(MQTT_CLIENT, nullptr, nullptr, "ttt/sys/server",
                     0, true, "{\"status\":\"offline\"}")) {
      Serial.println("OK");

      // Subscriptions
      mqtt.subscribe("ttt/game/new");
      mqtt.subscribe("ttt/game/+/move");
      mqtt.subscribe("ttt/lobby/announce");
      mqtt.subscribe("ttt/lobby/leave/#");
      mqtt.subscribe("ttt/lobby/invite/#");
      mqtt.subscribe("ttt/lobby/response/#");

      // Announce online (retained)
      mqtt.publish("ttt/sys/server", "{\"status\":\"online\"}", true);
      Serial.println("[MQTT] Subscriptions registered");
    } else {
      Serial.println("failed rc=" + String(mqtt.state()) + " retrying in
↪  3s");
      delay(3000);
    }
  }
}

//
//  Arduino entry points
//

void setup() {
  Serial.begin(115200);
  randomSeed(esp_random());

  connectWiFi();
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(onMessage);
  mqtt.setBufferSize(1024);
  connectMQTT();

  Serial.println("[TTT] Server ready");
}

void loop() {
  if (!mqtt.connected()) connectMQTT();
  mqtt.loop();
```

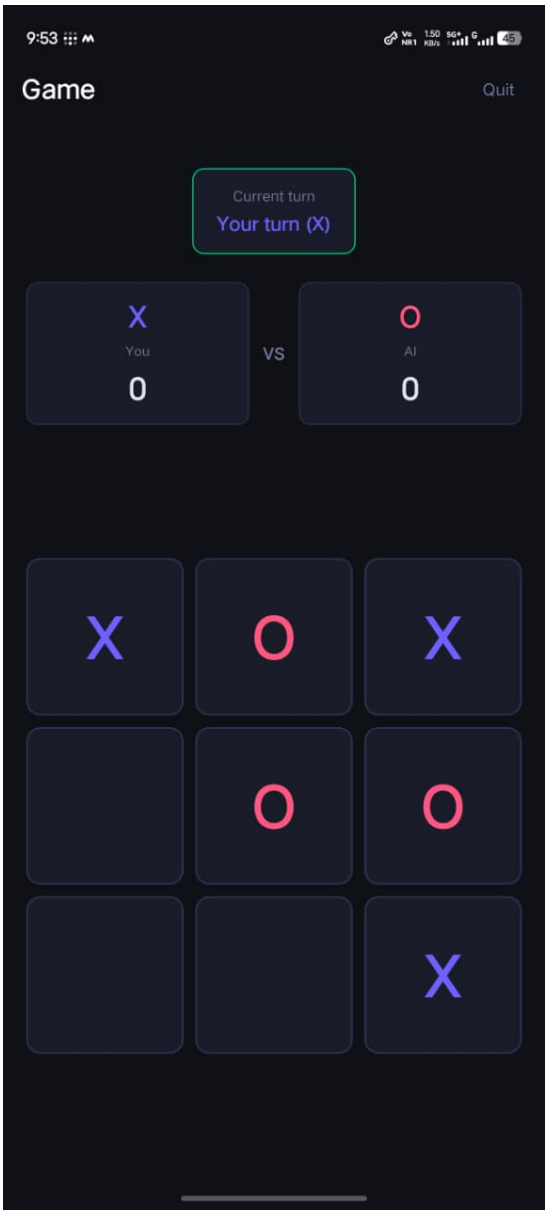
```
  // Heartbeat every 5 s
  if (millis() - lastHeartbeat >= 5000) {
    lastHeartbeat = millis();
    String hb = "{\"ts\":" + String(millis()) + ",\"sessions\":" +
                String(sessions.size()) + ",\"players\":" +
                String(players.size()) + "}";
    mqtt.publish("ttt/sys/heartbeat", hb.c_str());
  }
}
```

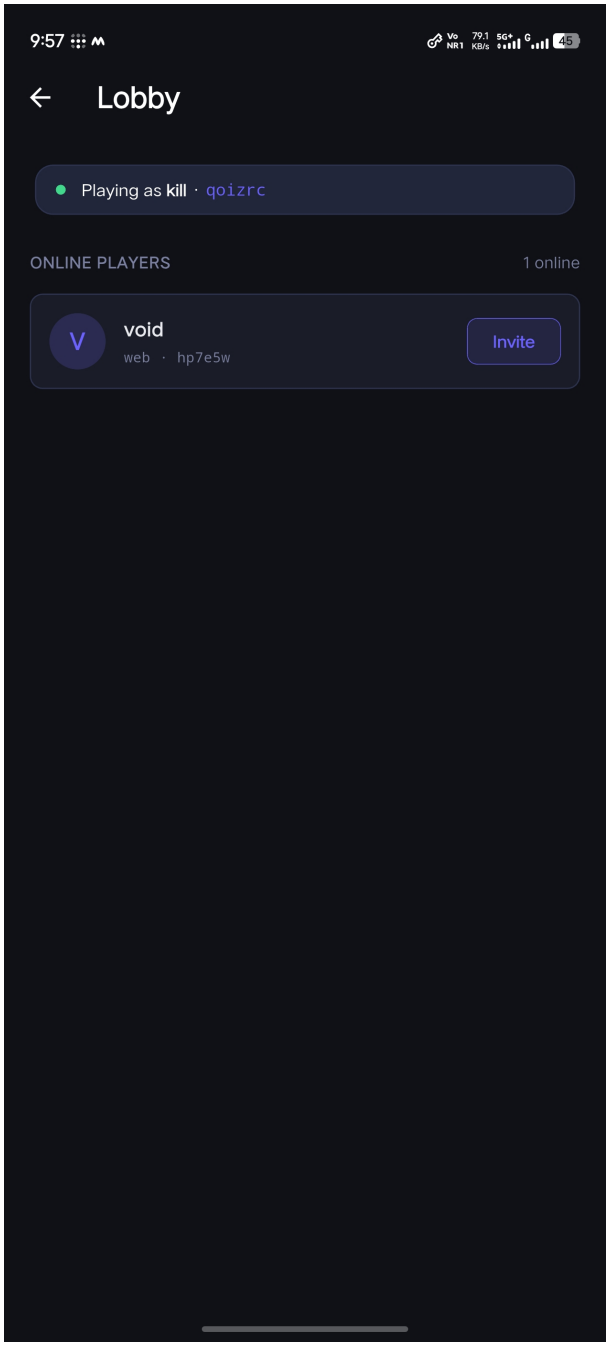
## JSON

```
{
  "version": 1,
  "author": "TTT MQTT Game",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-esp32-devkit-v1",
      "id": "esp",
      "top": 10,
      "left": 10,
      "attrs": {}
    }
  ],
  "connections": []
}
```

# APP


9:53
TicTacToe
Powered by MQTT · ESP32
Connected to broker
Your display name
krill
5/20
Game mode
Play vs AI
Human vs Human
Start Game
Player ID: qoizrcop


9:53
Game
Quit
Current turn
Your turn (X)
X
You
0
VS
O
AI
0
X
O
X
O
O
X


9:57
Lobby
Playing as kill · qoizrc
ONLINE PLAYERS
1 online
V
void
web · hp7e5w
Invite